

How Reader-Writer Locking Improves the Design

1. **Concurrent Reads:** With `std::shared_mutex`, multiple threads can hold a `std::shared_lock` simultaneously, allowing concurrent read access. This is ideal for scenarios where we frequently need to read the auction status but less frequently update it.
2. **Exclusive Writes:** Only one thread at a time can hold a `std::unique_lock` on the `std::shared_mutex`, ensuring exclusive write access. This is necessary for operations like placing a bid or ending the auction, which modify the auction state.
3. **Efficient Reporting:** Reader-writer locking allows the system to provide real-time reporting on the current state of active auctions without interfering with ongoing bidding activities.

Updated Design with `std::shared_mutex` for Reporting

Below is the updated code with `std::shared_mutex` added to allow concurrent reads. We'll also add a `reportCurrentState` function to display the status of active auctions.

```
#include <iostream>
#include <thread>
#include <mutex>
#include <shared_mutex>
#include <condition_variable>
#include <string>
#include <chrono>
#include <future>
#include <vector>
#include <map>

class Auction {
public:
    Auction(std::string item, int duration_seconds)
        : item_name(std::move(item)), duration(duration_seconds), highest_bid(0),
        auction_ended(false) {}

    // Function to place a bid on the item
    std::future<std::string> placeBid(int bid, const std::string& bidder, ThreadPool& thread_pool) {
        return thread_pool.enqueue([this, bid, bidder]() {
            std::unique_lock<std::shared_mutex> lock(bid_mutex); // Exclusive lock for modifying bid
            data

            if (auction_ended) {
                return "Auction for " + item_name + " has ended. No more bids allowed.";
            }
            if (bid > highest_bid) {
```

```

        highest_bid = bid;
        highest_bidder = bidder;
        std::string confirmation = "New highest bid for " + item_name + ": " +
std::to_string(bid) + " by " + bidder;
        std::cout << confirmation << std::endl;
        return confirmation;
    } else {
        std::string message = "Bid of " + std::to_string(bid) + " by " + bidder + " was too
low for " + item_name;
        std::cout << message << std::endl;
        return message;
    }
});
}

// Function to run the auction with a time limit
void startAuction(ThreadPool& thread_pool) {
    thread_pool.enqueue([this] {
        std::cout << "Starting auction for " << item_name << " lasting " << duration << "
seconds.\n";
        auto start_time = std::chrono::steady_clock::now();

        while (std::chrono::steady_clock::now() - start_time < std::chrono::seconds(duration)) {
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }

        {
            std::unique_lock<std::shared_mutex> lock(bid_mutex); // Exclusive lock to end
auction
            auction_ended = true;
        }
        cv.notify_all();
        std::cout << "Auction for " << item_name << " has ended.\n";
    });
}

// Function to get the final result of the auction
void waitForAuctionEnd(ThreadPool& thread_pool) {
    thread_pool.enqueue([this] {
        std::unique_lock<std::shared_mutex> lock(bid_mutex);
        cv.wait(lock, [this] { return auction_ended; });
        if (highest_bid > 0) {
            std::cout << "Auction result for " << item_name << ": Sold to " << highest_bidder
<< " for " << highest_bid << std::endl;

```

```

        } else {
            std::cout << "Auction result for " << item_name << ": No bids received.\n";
        }
    });
}

// Function to report the current state of the auction (for active reporting)
std::string reportStatus() const {
    std::shared_lock<std::shared_mutex> lock(bid_mutex); // Shared lock for reading data
    if (auction_ended) {
        return "Auction for " + item_name + " has ended.";
    } else {
        return "Current highest bid for " + item_name + " is " + std::to_string(highest_bid)
            + " by " + highest_bidder;
    }
}

private:
    std::string item_name;
    int duration;
    int highest_bid;
    std::string highest_bidder;
    bool auction_ended;
    mutable std::shared_mutex bid_mutex; // Shared mutex for reader-writer locking
    std::condition_variable_any cv; // Use condition_variable_any for shared_mutex support
};

// Function to report the state of all active auctions
void reportAllAuctions(const std::vector<std::shared_ptr<Auction>>& auctions) {
    for (const auto& auction : auctions) {
        std::cout << auction->reportStatus() << std::endl;
    }
}

// Simulate bidding by different users
void simulateBidding(Auction& auction, const std::string& bidder, int max_bid, ThreadPool&
thread_pool) {
    for (int bid = 10; bid <= max_bid; bid += 10) {
        std::future<std::string> result = auction.placeBid(bid, bidder, thread_pool);
        std::cout << "Confirmation for " << bidder << ": " << result.get() << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(150));
    }
}

```

```

int main() {
    ThreadPool thread_pool(4); // Create a thread pool with 4 threads

    auto auction1 = std::make_shared<Auction>("Painting", 5);
    auto auction2 = std::make_shared<Auction>("Antique Vase", 7);
    std::vector<std::shared_ptr<Auction>> auctions = {auction1, auction2};

    // Start auctions using the thread pool
    auction1->startAuction(thread_pool);
    auction2->startAuction(thread_pool);

    // Simulate bidding using the thread pool
    std::thread bidder1(simulateBidding, std::ref(*auction1), "Alice", 100, std::ref(thread_pool));
    std::thread bidder2(simulateBidding, std::ref(*auction1), "Bob", 80, std::ref(thread_pool));
    std::thread bidder3(simulateBidding, std::ref(*auction2), "Charlie", 90, std::ref(thread_pool));
    std::thread bidder4(simulateBidding, std::ref(*auction2), "Dave", 120, std::ref(thread_pool));

    // Periodic reporting of all active auctions
    std::thread reporter([&auctions] {
        while (true) {
            std::this_thread::sleep_for(std::chrono::seconds(1)); // Report every second
            reportAllAuctions(auctions);
        }
    });

    // Wait for auctions to end and display results
    auction1->waitForAuctionEnd(thread_pool);
    auction2->waitForAuctionEnd(thread_pool);

    // Join all threads
    bidder1.join();
    bidder2.join();
    bidder3.join();
    bidder4.join();
    reporter.detach(); // Detach reporter as it's a continuous task

    return 0;
}

```

Explanation of Changes

1. Reader-Writer Locking with `std::shared_mutex`:

- We've replaced the `std::mutex` in the `Auction` class with `std::shared_mutex` (assigned to `bid_mutex`) to allow multiple threads to read the auction state concurrently without blocking each other.

2. `std::shared_lock` for Read-Only Access:

- The `reportStatus` function acquires a `std::shared_lock` on `bid_mutex`, allowing it to read the auction data without blocking other readers. This allows the reporting thread to call `reportStatus` concurrently with other readers.

3. `std::unique_lock` for Exclusive Write Access:

- `placeBid`, `startAuction`, and `waitForAuctionEnd` still use `std::unique_lock` for exclusive write access, ensuring that these functions can modify the auction state safely.

4. New `reportAllAuctions` Function:

- This function iterates over a list of active auctions, calling `reportStatus` on each one to display the current highest bid and bidder without blocking bidding threads.
- We launch this function in a separate `reporter` thread that runs continuously to update the auction status every second.

Summary

Adding `std::shared_mutex`, `std::shared_lock`, and `std::unique_lock` allows us to implement efficient read access, which is essential for reporting on active auctions without interfering with ongoing bidding. This design allows multiple threads to read auction data concurrently while ensuring exclusive access for modifications, making the application more scalable and responsive for real-time updates and reporting.